

数字图像处理作业

电信 2303-樊博文-8207230322

目录

作业一、用百度识图识别校内的一朵花.....	2
作业二、在 matlab 中用函数 imread、imshow 读入并显示一幅图	3
作业三、在 matlab 中对 lena 图进行右移 5 个像素、下移 10 个像素处理.....	4
作业四、图像逆时针选择 45° ，编程实现	6
作业五、编写一个快速二维傅里叶变换，分析一幅图的幅值、相位角	9
作业六、matlab 中编程小波变换实现 lena 人像与反变换.....	12
作业七、用 3sigma 法则设计一个图像增强算法	15
作业八、试设计一个高斯带通滤波器，对一个图像进行增强（要有 f、要标注不同参数下的 输出效果）	19
作业九、在 lena 图生成三种以上的噪声图.....	23
作业十、模板匹配法实现左侧到右侧的结果.....	28
作业十一、自动找分隔线切图 + 计算 MSE/PSNR/SSIM 计算一下校园图两幅图和标准图的指 标	29
作业十二、用 sobel 算子和 LoG 算子设计二个程序对 lena 图进行边缘检测	33
作业十三、已知原始图，画出 T=5 的矩阵.....	39
作业十四、腐蚀膨胀.....	40
作业十五、求 C 和 D 的欧拉数	42

作业一、用百度识图识别校内的一朵花



作业二、在 matlab 中用函数 imread、imshow 读入并显示一幅图

% 作业二：在 MATLAB 中用 imread、imshow 读入并显示一幅图

```
clc;  
clear;  
close all;
```

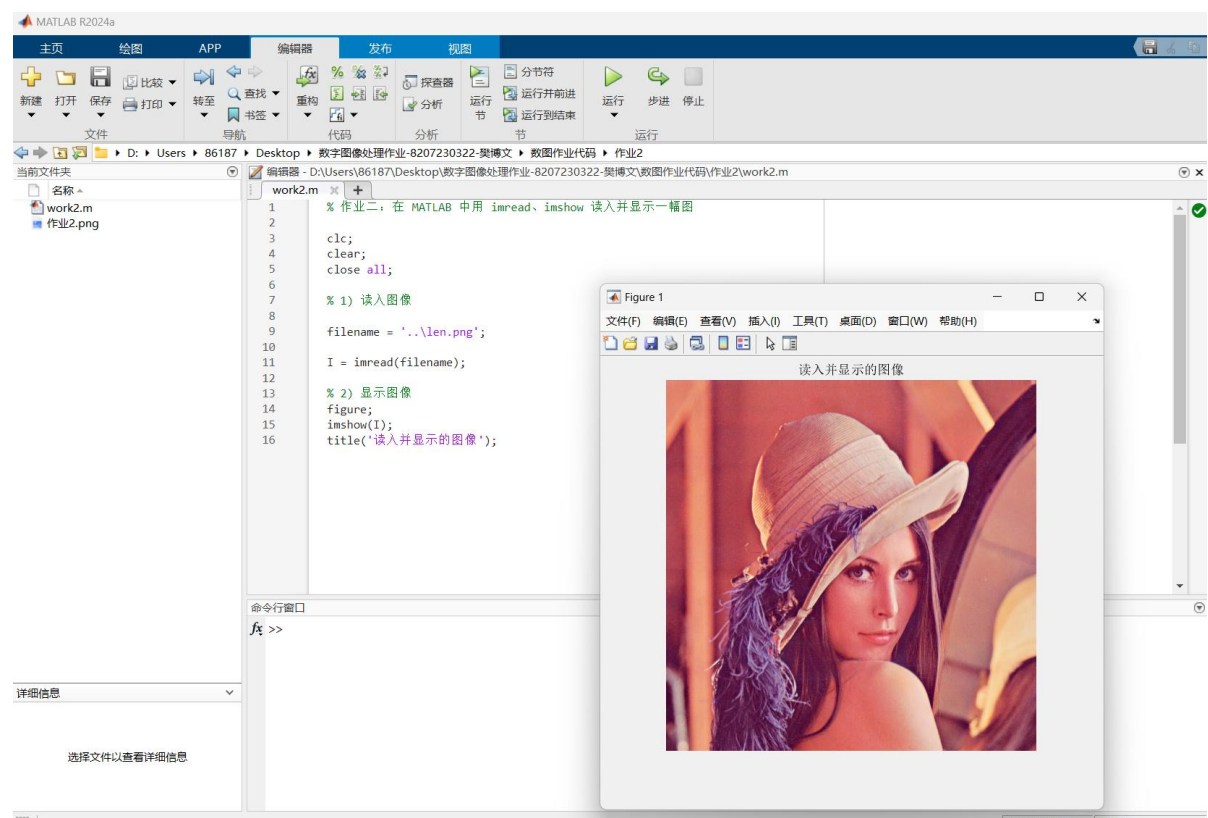
% 1) 读入图像

```
filename = '..\len.png';
```

```
I = imread(filename);
```

% 2) 显示图像

```
figure;  
imshow(I);  
title('读入并显示的图像');
```



作业三、在 matlab 中对 lena 图进行右移 5 个像素、下移 10 个像素处理

% 作业 3: 图像平移

clc; clear; close all;

f = '..\lena.png';

I = imread(f);

dx = 5; % 右移

dy = 10; % 下移

J = myShift(I, dx, dy);

imshow(J);

title('右移 5 像素, 下移 10 像素');

function J = myShift(I, dx, dy)

% dx>0 右移, dx<0 左移; dy>0 下移, dy<0 上移

[h,w,c] = size(I);

J = zeros(h,w,c, 'like', I);

for y = 1:h

for x = 1:w

ys = y - dy; % 反向映射: 目标(y,x)来自原图(ys,xs)

xs = x - dx;

if ys >= 1 && ys <= h && xs >= 1 && xs <= w

for k = 1:c

J(y,x,k) = I(ys,xs,k);

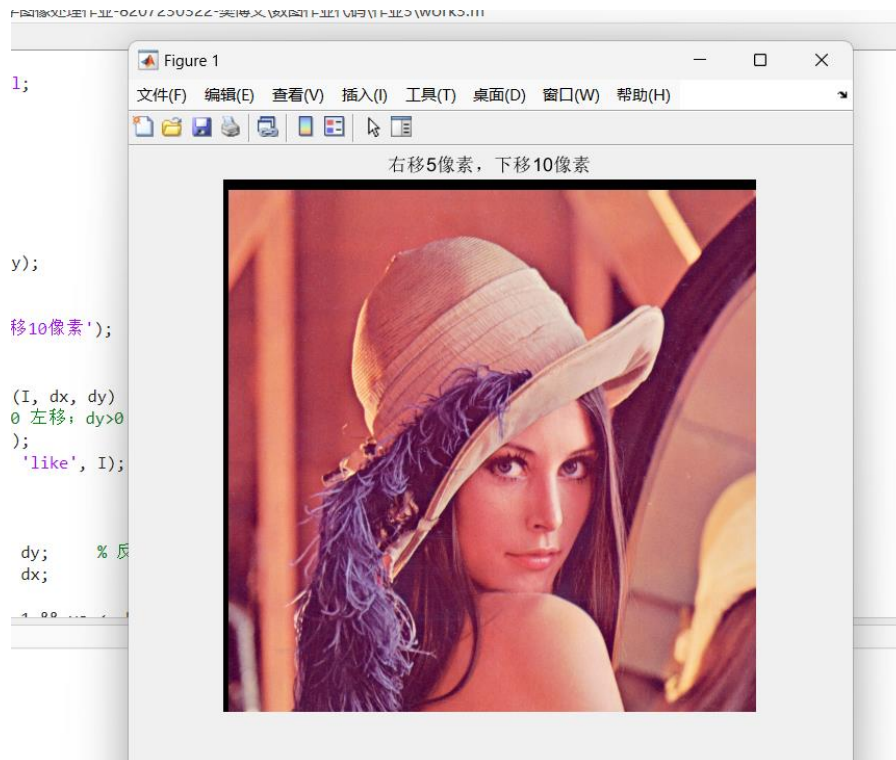
end

end

end

end

end



作业四、图像逆时针选择 45° ，编程实现

```
clc; clear; close all;

filename = '..\len.png';
I = imread(filename);

theta = 45;
rad = deg2rad(theta);

I = im2double(I);
[H, W, C] = size(I);

% 原图中心
cx = (W+1)/2;
cy = (H+1)/2;

corners = [1,1; W,1; 1,H; W,H];
x = corners(:,1) - cx;
y = corners(:,2) - cy;

xr = x*cos(rad) - y*sin(rad);
yr = x*sin(rad) + y*cos(rad);

minx = min(xr); maxx = max(xr);
miny = min(yr); maxy = max(yr);

W2 = ceil(maxx - minx + 1);
H2 = ceil(maxy - miny + 1);

% 新图中心
cx2 = (W2+1)/2;
cy2 = (H2+1)/2;

J = zeros(H2, W2, C);

% 输出图坐标
[XX, YY] = meshgrid(1:W2, 1:H2);
x2 = XX - cx2;
y2 = YY - cy2;

% 逆变换（映射回原图）
x1 = x2*cos(rad) + y2*sin(rad);
```

```

y1 = -x2*sin(rad) + y2*cos(rad);

Xsrc = x1 + cx;
Ysrc = y1 + cy;

% 双线性插值
for k = 1:C
    J(:, :, k) = bilinearSample(I(:, :, k), Xsrc, Ysrc);
end

J8 = im2uint8(J);

figure;
imshow(J8);
title('逆时针旋转 45°');

% ===== 双线性采样函数 =====
function out = bilinearSample(img, X, Y)
    [H, W] = size(img);
    out = zeros(size(X));

    valid = (X >= 1) & (X <= W) & (Y >= 1) & (Y <= H);
    Xv = X(valid); Yv = Y(valid);

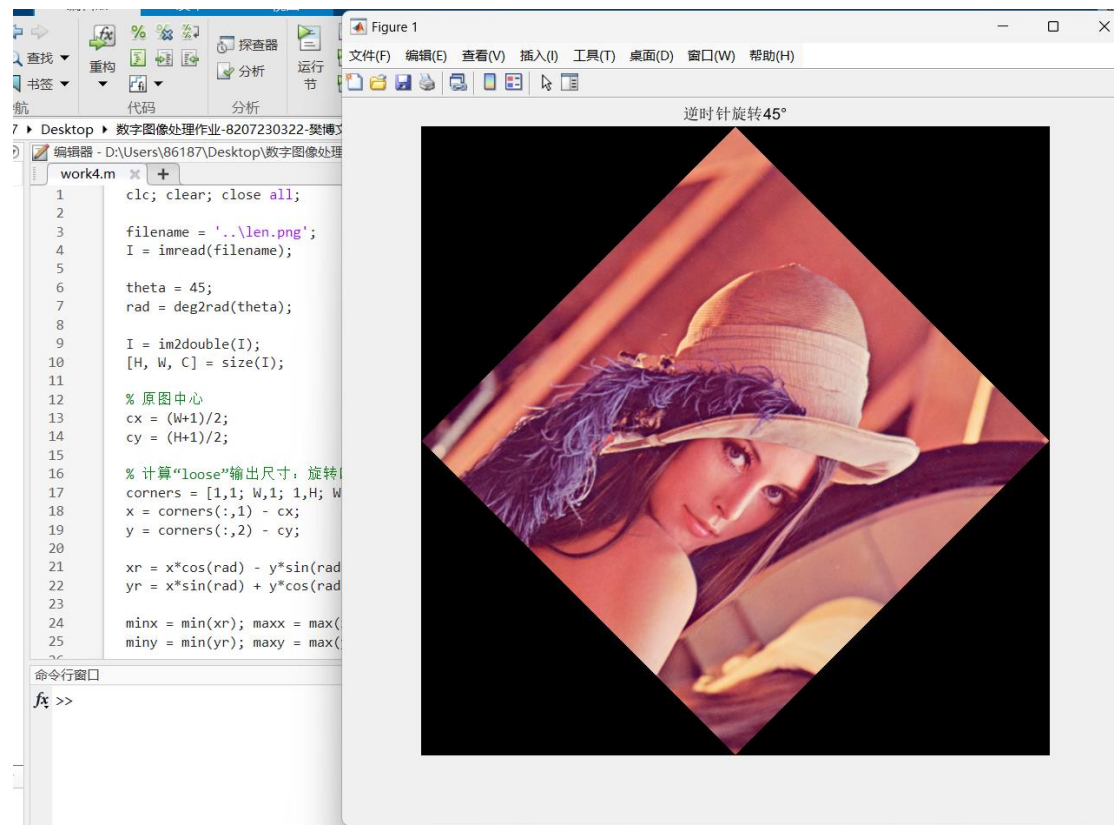
    x1 = floor(Xv); x2 = min(x1 + 1, W);
    y1 = floor(Yv); y2 = min(y1 + 1, H);

    dx = Xv - x1;
    dy = Yv - y1;

    Ia = img(sub2ind([H,W], y1, x1));
    Ib = img(sub2ind([H,W], y1, x2));
    Ic = img(sub2ind([H,W], y2, x1));
    Id = img(sub2ind([H,W], y2, x2));

    out(valid) = (1-dx).*(1-dy).*Ia + dx.*(1-dy).*Ib + (1-dx).*dy.*Ic +
    dx.*dy.*Id;
end

```



作业五、编写一个快速二维傅里叶变换，分析一幅图的幅值、相位角

```
clc; clear; close all;

f = '..\len.png';
img = imread(f);
if ndims(img)==3, img = rgb2gray(img); end
img = im2double(img);

% 补零到 2 的幂
[m,n] = size(img);
p = 2^nextpow2(m);
q = 2^nextpow2(n);
x = zeros(p,q);
x(1:m,1:n) = img;

% 二维 FFT: 先行后列
F = myfft2(x);
Fs = fftshift(F);

A = log(1 + abs(Fs));
P = angle(Fs);
P = (P + pi) / (2*pi);

figure; imshow(A,[]); title('幅值');
figure; imshow(P,[]); title('相位');

function Y = myfft2(X)
% 2D FFT = 对每行做 1D FFT, 再对每列做 1D FFT
[h,w] = size(X);
Y = complex(zeros(h,w));
for i = 1:h
    Y(i,:) = myfft1(X(i,:));
end
for j = 1:w
    Y(:,j) = myfft1(Y(:,j)).';
end
end

function X = myfft1(x)

x = complex(x(:));
N = length(x);
```

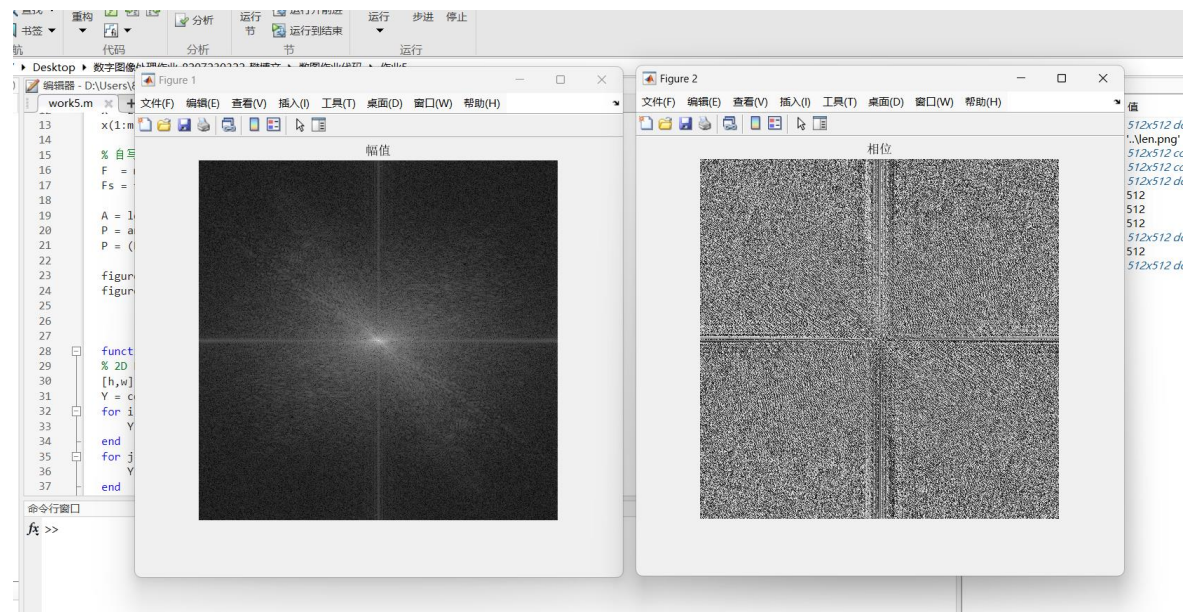
```

X = x(bitrev_idx(N));

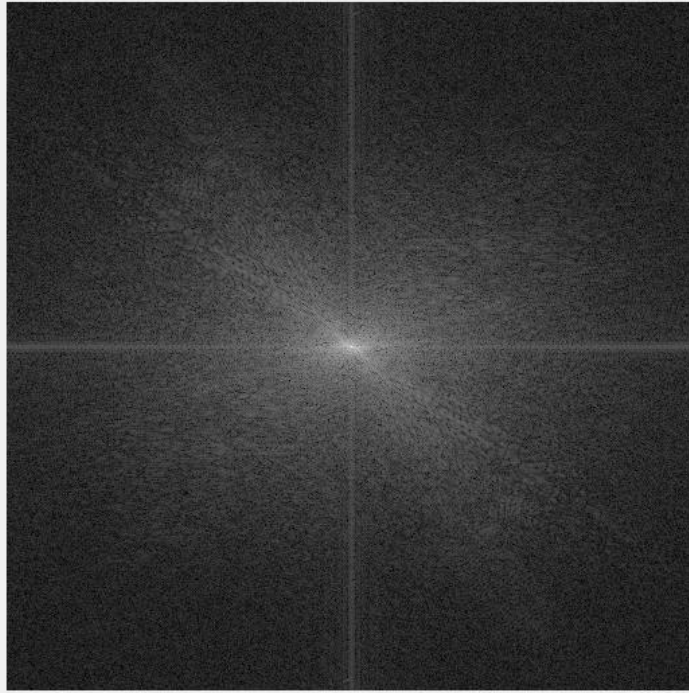
m = 2;
while m <= N
    h = m/2;
    W = exp(-2i*pi*(0:h-1)/m).'; % 旋转因子
    for k = 1:m:N % 蝶形运算
        u = X(k:k+h-1);
        t = W .* X(k+h:k+m-1);
        X(k:k+h-1) = u + t;
        X(k+h:k+m-1) = u - t;
    end
    m = m*2;
end
X = X.';
end
function idx = bitrev_idx(N)

b = log2(N);
idx = zeros(1,N);
for i = 0:N-1
    r = 0; v = i;
    for t = 1:b
        r = bitshift(r,1) + bitand(v,1);
        v = bitshift(v,-1);
    end
    idx(i+1) = r + 1;
end
end

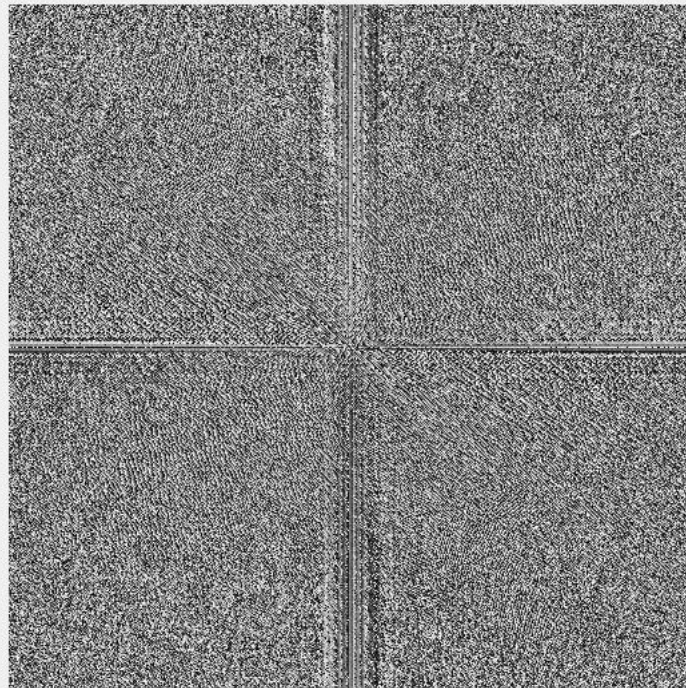
```



幅值



相位



作业六、matlab 中编程小波变换实现 lena 人像与反变换

```
clc; clear; close all;

f = '..\len.png';
img = imread(f);
if ndims(img)==3, img = uint8(0.299*img(:,:,1) + 0.587*img(:,:,2) +
0.114*img(:,:,3)); end
img = double(img) / 255;

% 2D 小波分解
[LL, LH, HL, HH] = haar_dwt2(img);
figure;
subplot(2,2,1); imagesc(LL); axis image off; colormap gray; title('LL');
subplot(2,2,2); imagesc(LH); axis image off; colormap gray; title('LH');
subplot(2,2,3); imagesc(HL); axis image off; colormap gray; title('HL');
subplot(2,2,4); imagesc(HH); axis image off; colormap gray; title('HH');
% 反变换重建
rec = haar_idwt2(LL, LH, HL, HH);

figure;
imagesc(rec); axis image off; colormap gray; title('反变换重建');

function [LL, LH, HL, HH] = haar_dwt2(x)
    [m,n] = size(x);

    if mod(m,2)==1, x(end+1,:) = x(end,:); m = m+1; end
    if mod(n,2)==1, x(:,end+1) = x(:,end); n = n+1; end

    s = sqrt(2);

    a = x(:,1:2:end);
    b = x(:,2:2:end);
    Lr = (a + b) / s;
    Hr = (a - b) / s;
    a = Lr(1:2:end,:);
    b = Lr(2:2:end,:);
    LL = (a + b) / s;
    LH = (a - b) / s;
    a = Hr(1:2:end,:);
    b = Hr(2:2:end,:);
```

```

    HL = (a + b) / s; % 高行低列
    HH = (a - b) / s; % 高行高列
end

function x = haar_idwt2(LL, LH, HL, HH)
    s = sqrt(2);

    Lr = inv_haar_col(LL, LH, s);
    Hr = inv_haar_col(HL, HH, s);

    x = inv_haar_row(Lr, Hr, s);
end

function y = inv_haar_col(L, H, s)

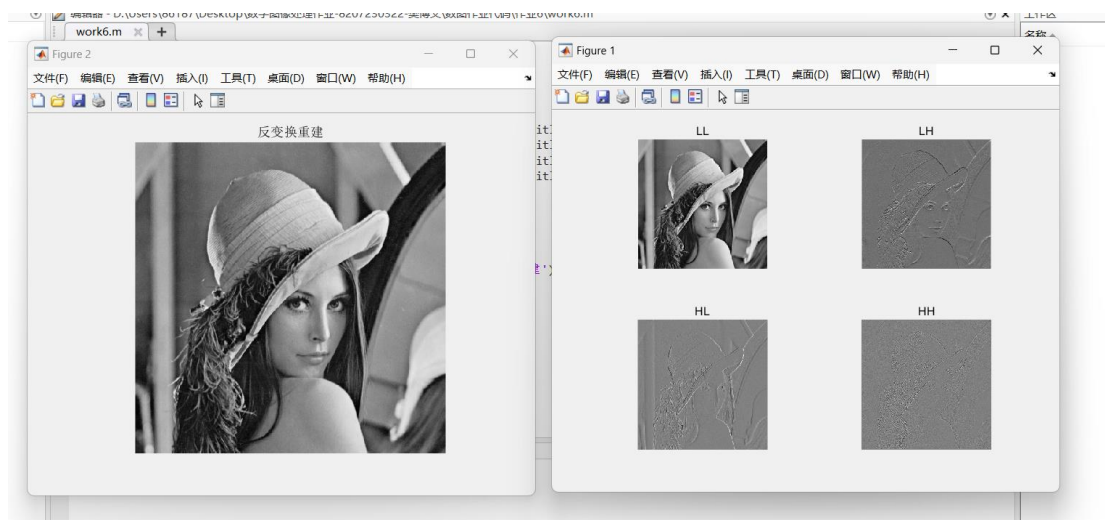
    [k, w] = size(L);
    y = zeros(2*k, w);
    y(1:2:end,:) = (L + H) / s;
    y(2:2:end,:) = (L - H) / s;
end

function y = inv_haar_row(L, H, s)

    [h, k] = size(L);
    y = zeros(h, 2*k);

    y(:,1:2:end) = (L + H) / s;
    y(:,2:2:end) = (L - H) / s;
end

```



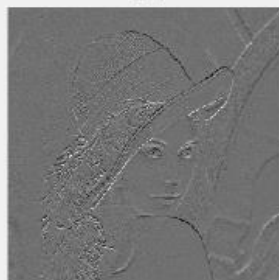
反变换重建



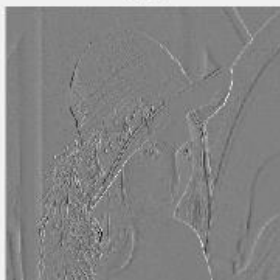
LL



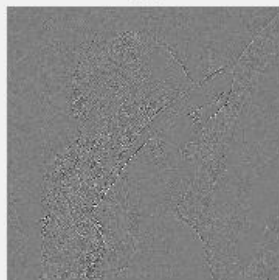
LH



HL



HH



作业七、用 3sigma 法则设计一个图像增强算法

```
clc; clear; close all;

f = 'test.png';
img = imread(f);

out = enhance3sigma(img);

figure;
cmp = cat(2, img, out); % 横向拼接
imshow(cmp);
title('左: 原图 右: 3\sigma 增强');

function out = enhance3sigma(img)
    cls = class(img);

    x = to01(img);

    if ndims(x) == 2
        y = stretch3sigma_gray(x);
        out = from01(y, cls);
        return;
    end

    R = x(:,:,1); G = x(:,:,2); B = x(:,:,3);
    Y = 0.299*R + 0.587*G + 0.114*B;

    Ye = stretch3sigma_gray(Y);

    epsv = 1e-8;
    ratio = Ye ./ (Y + epsv);

    Re = clamp01(R .* ratio);
    Ge = clamp01(G .* ratio);
    Be = clamp01(B .* ratio);

    out = from01(cat(3, Re, Ge, Be), cls);
end

function y = stretch3sigma_gray(x)
```

```

% x in [0,1]
[h,w] = size(x);
n = h*w;

% 计算均值
s = 0;
for i = 1:h
    for j = 1:w
        s = s + x(i,j);
    end
end
mu = s / n;

% 计算标准差
s2 = 0;
for i = 1:h
    for j = 1:w
        d = x(i,j) - mu;
        s2 = s2 + d*d;
    end
end
sigma = sqrt(s2 / n);

a = mu - 3*sigma;
b = mu + 3*sigma;

% 防止阈值越界
if a < 0, a = 0; end
if b > 1, b = 1; end

% 防止除 0
if b <= a
    y = x;
    return;
end

% clip + stretch
y = zeros(h,w);
for i = 1:h
    for j = 1:w
        v = x(i,j);
        if v < a, v = a; end
        if v > b, v = b; end
        y(i,j) = (v - a) / (b - a);
    end
end

```

```

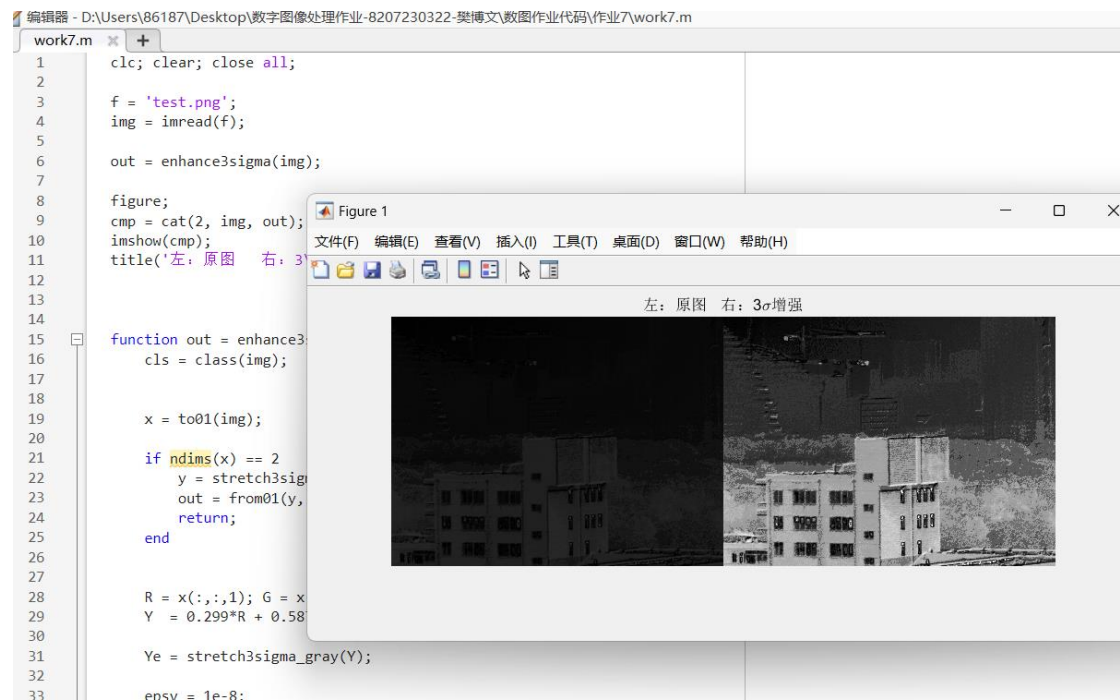
        end
    end
end

function x = to01(img)
    if isa(img, 'uint8')
        x = double(img) / 255;
    elseif isa(img, 'uint16')
        x = double(img) / 65535;
    else
        x = double(img);
        % 若输入本来不是[0,1], 简单归一化到[0,1]
        mn = min(x(:)); mx = max(x(:));
        if mx > mn
            x = (x - mn) / (mx - mn);
        else
            x = zeros(size(x));
        end
    end
    x = clamp01(x);
end

function out = from01(x, cls)
    x = clamp01(x);
    if strcmp(cls, 'uint8')
        out = uint8(round(x * 255));
    elseif strcmp(cls, 'uint16')
        out = uint16(round(x * 65535));
    else
        out = cast(x, cls);
    end
end

function y = clamp01(x)
    y = x;
    y(y < 0) = 0;
    y(y > 1) = 1;
end

```



作业八、试设计一个高斯带通滤波器，对一个图像进行增强（要有 f' 、要标注不同参数下的输出效果）

```
clc; clear; close all;

% 输入图像
fpath = '..\test.png';
f = imread(fpath);
if ndims(f) == 3
    f = uint8(0.299*f(:,:,1) + 0.587*f(:,:,2) + 0.114*f(:,:,3));
end
f = double(f) / 255;

[M,N] = size(f);
D = freqRadius(M,N);

F = fftshift(fft2(f));

mnd = min(M,N);
params = [
    0.02*mnd    0.08*mnd    1.0
    0.04*mnd    0.12*mnd    1.0
    0.06*mnd    0.16*mnd    1.0
    0.04*mnd    0.12*mnd    1.8
];

K = size(params,1);

% 展示：每组参数两张图（上：带通分量 下：增强结果  $f'$ ）
figure;
tiledlayout(2, K, 'Padding', 'compact', 'TileSpacing', 'compact');

for i = 1:K
    s1 = params(i,1);
    s2 = params(i,2);
    a = params(i,3);

    H = gaussBP(D, s1, s2);          % 高斯带通
    G = F .* H;
    g = real(ifft2(ifftshift(G)));
```

```

    fp = f + a*g;
    fp = clamp01(fp);

    gv = vis01(g);

    nexttile(i);
    imagesc(gv); axis image off; colormap gray;
    title(sprintf('GBP 分量  \sigma1=%.0f \sigma2=%.0f', s1, s2));

    nexttile(K+i);
    imagesc(fp); axis image off; colormap gray;
    title(sprintf('f'  \alpha=%.1f', a));
end

figure;
tiledlayout(1, K, 'Padding','compact','TileSpacing','compact');
for i = 1:K
    s1 = params(i,1); s2 = params(i,2);
    H = gaussBP(D, s1, s2);
    nexttile(i);
    imagesc(H); axis image off; colormap gray;
    title(sprintf('H(u,v) \sigma1=%.0f \sigma2=%.0f', s1, s2));
end

function D = freqRadius(M,N)

    u = (-floor(M/2)):(ceil(M/2)-1);
    v = (-floor(N/2)):(ceil(N/2)-1);
    [V,U] = meshgrid(v,u);
    D = sqrt(U.^2 + V.^2);
end

function H = gaussBP(D, s1, s2)

    H2 = exp(-(D.^2) / (2*s2*s2));
    H1 = exp(-(D.^2) / (2*s1*s1));
    H = H2 - H1;
end

function y = clamp01(x)

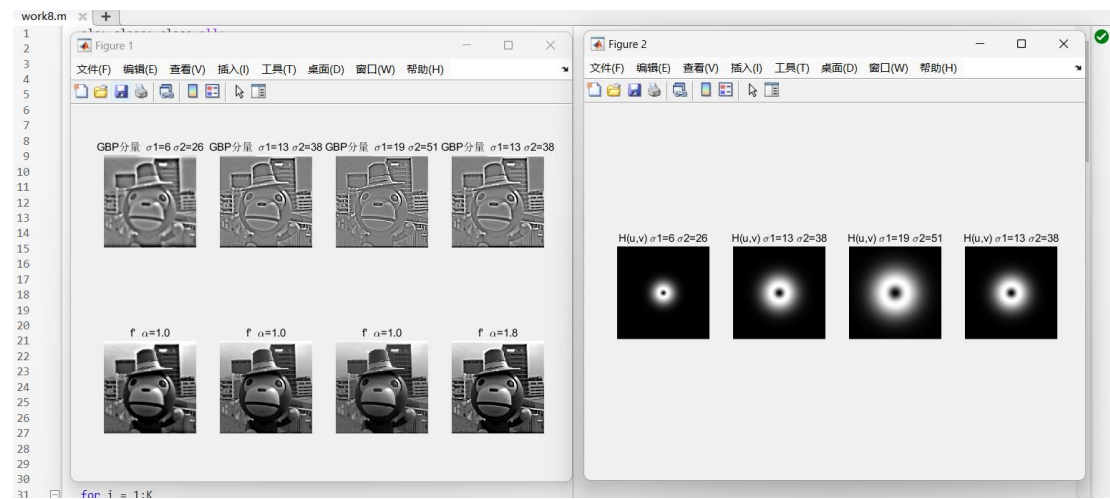
```

```

y = x;
y(y<0) = 0;
y(y>1) = 1;
end

function y = vis01(x)
% 把带通分量可视化到[0,1]: 按均值 $\pm 3\sigma$  压缩显示
mu = mean(x(:));
sg = std(x(:));
a = mu - 3*sg; b = mu + 3*sg;
if b <= a
    y = zeros(size(x));
    return;
end
y = (x - a) / (b - a);
y = clamp01(y);
end

```



GBP分量 $\sigma_1=6$ $\sigma_2=26$ GBP分量 $\sigma_1=13$ $\sigma_2=38$ GBP分量 $\sigma_1=19$ $\sigma_2=51$ GBP分量 $\sigma_1=13$ $\sigma_2=38$



$f'_{\alpha=1.0}$



$f'_{\alpha=1.0}$



$f'_{\alpha=1.0}$



$f'_{\alpha=1.8}$



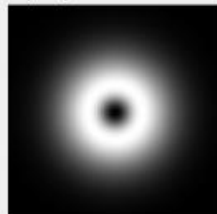
$H(u,v)$ $\sigma_1=6$ $\sigma_2=26$



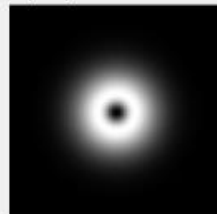
$H(u,v)$ $\sigma_1=13$ $\sigma_2=38$



$H(u,v)$ $\sigma_1=19$ $\sigma_2=51$



$H(u,v)$ $\sigma_1=13$ $\sigma_2=38$



作业九、在 lena 图生成三种以上的噪声图

```
clc; clear; close all;

f = '..\lena.png';
img = imread(f);

% 转灰度
if ndims(img)==3
    img = uint8(0.299*img(:,:,1) + 0.587*img(:,:,2) + 0.114*img(:,:,3));
end
x = double(img)/255;

% 参数
sig = 0.08;    % 高斯噪声标准差
peak = 30;     % 泊松噪声
d = 0.03;     % 椒盐噪声密度
A = 0.12;     % 均匀噪声幅度

g = addGaussian(x, sig);
p = addPoisson(x, peak);
sp = addSaltPepper(x, d);
u = addUniform(x, A);

% 显示并标注参数
figure;
tiledlayout(2,2,'Padding','compact','TileSpacing','compact');

nexttile; imagesc(g); axis image off; colormap gray;
title(sprintf('高斯噪声 ', sig));

nexttile; imagesc(p); axis image off; colormap gray;
title(sprintf('泊松噪声 ', peak));

nexttile; imagesc(sp); axis image off; colormap gray;
title(sprintf('椒盐噪声', d));

nexttile; imagesc(u); axis image off; colormap gray;
title(sprintf('均匀噪声', A));

% 保存
imwrite(uint8(round(g*255)), 'lena_gaussian.png');
imwrite(uint8(round(p*255)), 'lena_poisson.png');
```

```

imwrite(uint8(round(sp*255)), 'lena_saltpepper.png');
imwrite(uint8(round(u*255)), 'lena_uniform.png');

```

```

function y = addGaussian(x, sigma)
    y = x + sigma*randn(size(x));
    y = clamp01(y);
end

```

```

function y = addUniform(x, A)
    y = x + (2*rand(size(x)) - 1)*A;    % [-A, +A]
    y = clamp01(y);
end

```

```

function y = addSaltPepper(x, d)
    y = x;
    r = rand(size(x));
    y(r < d/2) = 0;           % pepper
    y(r > 1-d/2) = 1;        % salt
    y = clamp01(y);
end

```

```

function y = addPoisson(x, peak)

    lam = x * peak;
    y = zeros(size(x));

    small = lam < 30;
    y(small) = poisson_knuth(lam(small)) / peak;

    big = ~small;
    if any(big(:))
        lb = lam(big);
        k = round(lb + sqrt(lb).*randn(size(lb))); % 近似 Poisson
        k(k<0) = 0;
        y(big) = k / peak;
    end

    y = clamp01(y);
end

```

```

function k = poisson_knuth(lam)

```

```

k = zeros(size(lam));
for idx = 1:numel(lam)
    L = exp(-lam(idx));
    p = 1;
    c = 0;
    while p > L
        c = c + 1;
        p = p * rand();
    end
    k(idx) = c - 1;
end
end

function y = clamp01(x)
    y = x;
    y(y<0) = 0;
    y(y>1) = 1;
end

```

高斯噪声



泊松噪声



椒盐噪声



均匀噪声







作业十、模板匹配法实现左侧到右侧的结果

均值滤波：对每个窗口中的所有像素求平均值作为滤波结果， 3×3 大小的均值滤波卷积核为：

$$H_0 = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Q：边界像素怎么处理？

1	2	1	4	3
1	2	2	3	4
5	7	6	8	9
5	7	6	8	8
5	6	7	8	9



1	2	1	4	3
1	3	4	4	4
5	4	5	6	9
5	6	7	8	8
5	6	7	8	9

Handwritten notes on a digital whiteboard showing the calculation of the mean filter result for the input image.

Input image (5x5 grid):

1	2	1	4	3
1	2	2	3	4
5	7	6	8	9
5	7	6	8	8
5	6	7	8	9

Convolution kernel (3×3):

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Calculation for the center pixel (row 2, column 2):

$$(1 + 2 + 1 + 1 + 2 + 2 + 5 + 7 + 6) \div 9 = 3$$

Calculation for the center pixel (row 3, column 2):

$$(1 + 2 + 2 + 5 + 7 + 6 + 5 + 7 + 6) \div 9 = 4$$

Output image (5x5 grid):

1	2	1	4	3
1	3	4	4	4
5	4	5	6	9
5	6	7	8	8
5	6	7	8	9

Other notes: 其他类似. 边缘不处理

作业十一、自动找分隔线切图 + 计算 MSE/PSNR/SSIM 计算一下校园图两幅图和标准图的

指标

```
%% 作业 11: 自动找分隔线切图 + 计算 MSE/PSNR/SSIM
clc; clear; close all;

imgPath = 'work11.png';
I = imread(imgPath);
if size(I,3) == 1, I = repmat(I,[1 1 3]); end

[H,W,~] = size(I);

[s1, s2] = findSeparators(I);

L = I(:, 1:s1-1, :);
M = I(:, s1:s2-1, :);
R = I(:, s2:end, :);

L = imresize(L, [size(R,1), size(R,2)]);
M = imresize(M, [size(R,1), size(R,2)]);

[mseL, psnrL, ssimL] = calcMetrics(L, R);
[mseM, psnrM, ssimM] = calcMetrics(M, R);

fprintf('自动分隔位置: s1=%d, s2=%d (图像宽度 W=%d)\n', s1, s2, W);
fprintf('左图 vs 右标准: MSE=%.6f, PSNR=%.6f dB, SSIM=%.6f\n', mseL, psnrL, ssimL);
fprintf('中图 vs 右标准: MSE=%.6f, PSNR=%.6f dB, SSIM=%.6f\n', mseM, psnrM, ssimM);

% 切割结果
figure; imshow(L); title('Left (resized)');
figure; imshow(M); title('Middle (resized)');
figure; imshow(R); title('Right (Reference)');
```

```

function [s1, s2] = findSeparators(I)

    [~,W,~] = size(I);

    G = rgb2gray(I);
    colMean = mean(double(G), 1);
    colMean = colMean / max(colMean + eps);

    thr = max(0.92, prctile(colMean, 99));
    bright = colMean >= thr;

    edge = max(5, round(0.03 * W));
    bright(1:edge) = false;
    bright(end-edge+1:end) = false;

    idx = find(bright);
    if isempty(idx)

        s1 = floor(W/3);
        s2 = floor(2*W/3);
        return;
    end

    breaks = [true, diff(idx) > 1, true];
    cutPos = find(breaks);
    segs = {};
    for k = 1:numel(cutPos)-1
        seg = idx(cutPos(k):cutPos(k+1)-1);
        segs{end+1} = seg;
    end

    cand = [];
    for i = 1:numel(segs)
        seg = segs{i};
        w = numel(seg);
        c = mean(seg);

        if w <= round(0.05*W)
            cand = [cand; w, c];
        end
    end
end

```

```

end

if size(cand,1) < 2

    s1 = floor(W/3);
    s2 = floor(2*W/3);
    return;
end

t1 = W/3; t2 = 2*W/3;

[~, i1] = min(abs(cand(:,2) - t1));
c1 = cand(i1,2);

dist2 = abs(cand(:,2) - t2) + 1e6*(abs(cand(:,2) - c1) < 0.1*W);
[~, i2] = min(dist2);
c2 = cand(i2,2);

s = sort([round(c1), round(c2)]);
s1 = s(1);
s2 = s(2);

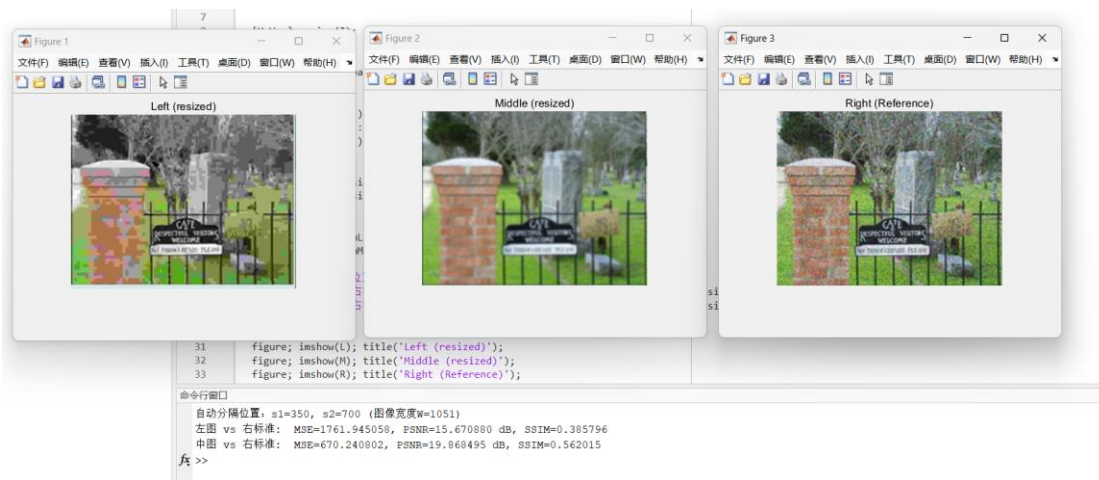
s1 = max(2, min(W-2, s1));
s2 = max(s1+2, min(W-1, s2));
end

function [mseVal, psnrVal, ssimVal] = calcMetrics(testImg, refImg)

if ~isa(testImg, 'uint8'), testImg = im2uint8(testImg); end
if ~isa(refImg, 'uint8'), refImg = im2uint8(refImg); end

mseVal = immse(testImg, refImg);
psnrVal = psnr(testImg, refImg);
ssimVal = ssim(testImg, refImg);
end

```



自动分隔位置: $s1=350$, $s2=700$ (图像宽度 $W=1051$)

左图 vs 右标准: $MSE=1761.945058$, $PSNR=15.670880$ dB, $SSIM=0.385796$

中图 vs 右标准: $MSE=670.240802$, $PSNR=19.868495$ dB, $SSIM=0.562015$

作业十二、用 **sobel** 算子和 **LoG** 算子设计二个程序对 **lena** 图进行边缘检测

```
clc; clear; close all;
f = '..\len.png';
img = imread(f);

% 转灰度并归一化到[0,1]
x = toGray01(img);

% Sobel 边缘
th = 0.25;
[mag, edge] = sobelEdge(x, th);

% 显示
figure('Color','w');
tiledlayout(1,3,'Padding','compact','TileSpacing','compact');

nexttile; imagesc(x); axis image off; colormap gray; title('原图(灰度)');
nexttile; imagesc(mag); axis image off; colormap gray; title('Sobel 幅值');
nexttile; imagesc(edge); axis image off; colormap gray;
title(sprintf('Sobel 边缘(阈值=%.2f)', th));

% 保存
imwrite(uint8(round(mag*255)), 'lena_sobel_mag.png');
imwrite(uint8(edge*255), 'lena_sobel_edge.png');

function x = toGray01(img)
    if ndims(img)==3
        img = 0.299*double(img(:,:,1)) + 0.587*double(img(:,:,2)) +
0.114*double(img(:,:,3));
    else
        img = double(img);
    end
    x = img / 255;
    x = clamp01(x);
end

function [mag, edge] = sobelEdge(x, th)
    % Sobel 核
    kx = [-1 0 1; -2 0 2; -1 0 1];
```

```

ky = [-1 -2 -1; 0 0 0; 1 2 1];

gx = myConv2(x, kx);
gy = myConv2(x, ky);

mag = sqrt(gx.^2 + gy.^2);

mag = mag / (max(mag(:)) + 1e-12);

edge = mag >= th; % 二值边缘
end

function y = myConv2(x, k)

[H,W] = size(x);
[kh,kw] = size(k);
ph = floor(kh/2);
pw = floor(kw/2);

xp = padReplicate(x, ph, pw);

y = zeros(H,W);
for r = 1:H
    for c = 1:W
        s = 0;
        for i = 1:kh
            for j = 1:kw
                s = s + xp(r+i-1, c+j-1) * k(i,j);
            end
        end
        y(r,c) = s;
    end
end
end

function xp = padReplicate(x, ph, pw)
[H,W] = size(x);
xp = zeros(H+2*ph, W+2*pw);

for r = 1:(H+2*ph)
    rr = r - ph;
    if rr < 1, rr = 1; end
    if rr > H, rr = H; end
    for c = 1:(W+2*pw)

```

```

        cc = c - pw;
        if cc < 1, cc = 1; end
        if cc > W, cc = W; end
        xp(r,c) = x(rr,cc);
    end
end
end

function y = clamp01(x)
    y = x;
    y(y<0) = 0;
    y(y>1) = 1;
end

```



```

clc; clear; close all;

f = '..\len.png';
img = imread(f);

% 转灰度并归一化
x = toGray01(img);

sigma = 1.4;
ksize = 9;
zc_th = 0.03;

% LoG 边缘
[resp, edge] = logEdge(x, sigma, ksize, zc_th);

% 显示

```

```

figure('Color','w');
tiledlayout(1,3,'Padding','compact','TileSpacing','compact');

nexttile; imagesc(x); axis image off; colormap gray; title('原图(灰度)');
nexttile; imagesc(resp); axis image off; colormap gray;
title(sprintf('LoG 响应(\\sigma=%.1f, k=%d)', sigma, ksize));
nexttile; imagesc(edge); axis image off; colormap gray;
title(sprintf('LoG 零交叉边缘(阈值=%.3f)', zc_th));

resp_vis = resp;
mn = min(resp_vis(:)); mx = max(resp_vis(:));
resp_vis = (resp_vis - mn) / (mx - mn + 1e-12);

imwrite(uint8(round(resp_vis*255)), 'lena_log_resp.png');
imwrite(uint8(edge*255), 'lena_log_edge.png');

function x = toGray01(img)
    if ndims(img)==3
        img = 0.299*double(img(:,:,1)) + 0.587*double(img(:,:,2)) +
0.114*double(img(:,:,3));
    else
        img = double(img);
    end
    x = img / 255;
    x = clamp01(x);
end

function [resp, edge] = logEdge(x, sigma, ksize, zc_th)
    k = makeLoGKernel(ksize, sigma);
    resp = myConv2(x, k);

    edge = zeroCrossing(resp, zc_th);
end

function k = makeLoGKernel(ksize, sigma)
    r = floor(ksize/2);
    s2 = sigma^2; s4 = s2^2;

    k = zeros(ksize, ksize);
    sumk = 0;

```

```

for y = -r:r
    for x = -r:r
        rr = x*x + y*y;
        val = ((rr - 2*s2)/s4) * exp(-rr/(2*s2));
        k(y+r+1, x+r+1) = val;
        sumk = sumk + val;
    end
end

k = k - sumk/(ksize*ksize);
end

function edge = zeroCrossing(resp, th)
    [H,W] = size(resp);
    edge = false(H,W);

    for r = 2:H-1
        for c = 2:W-1
            nb = resp(r-1:r+1, c-1:c+1);
            mn = min(nb(:));
            mx = max(nb(:));

            if (mn < 0) && (mx > 0) && ((mx - mn) >= th)
                edge(r,c) = true;
            end
        end
    end
end

function y = myConv2(x, k)
    [H,W] = size(x);
    [kh,kw] = size(k);
    ph = floor(kh/2);
    pw = floor(kw/2);

    xp = padReplicate(x, ph, pw);

    y = zeros(H,W);
    for r = 1:H
        for c = 1:W
            s = 0;
            for i = 1:kh
                for j = 1:kw

```

```

        s = s + xp(r+i-1, c+j-1) * k(i,j);
    end
end
y(r,c) = s;
end
end
end

```

```

function xp = padReplicate(x, ph, pw)
[H,W] = size(x);
xp = zeros(H+2*ph, W+2*pw);

for r = 1:(H+2*ph)
    rr = r - ph;
    if rr < 1, rr = 1; end
    if rr > H, rr = H; end
    for c = 1:(W+2*pw)
        cc = c - pw;
        if cc < 1, cc = 1; end
        if cc > W, cc = W; end
        xp(r,c) = x(rr,cc);
    end
end
end

```

```

function y = clamp01(x)
y = x;
y(y<0) = 0;
y(y>1) = 1;
end

```

原图(灰度)



LoG 响应($\sigma=1.4$, $k=9$)



LoG 零交叉边缘(阈值=0.030)



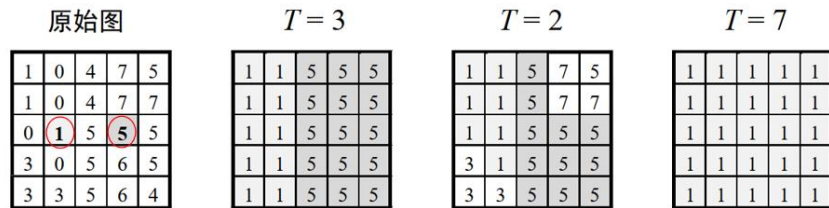
作业十三、已知原始图，画出 $T=5$ 的矩阵

图像分割 - 区域生长法



• 区域生长示例

➤ 种子点: 1和5

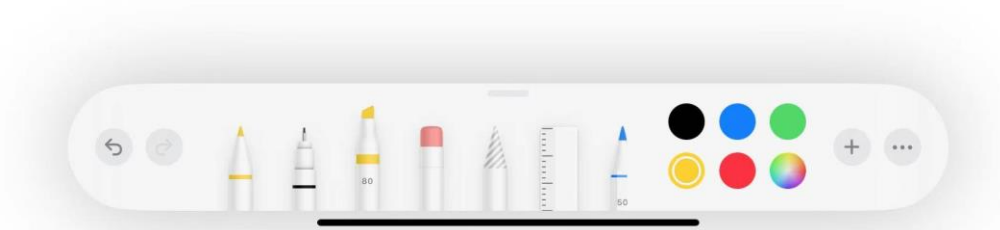
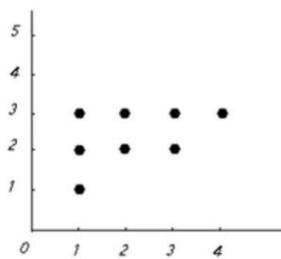
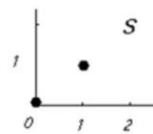
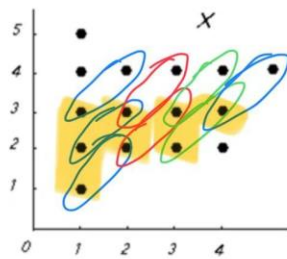
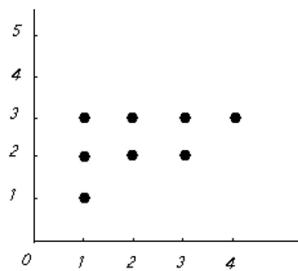
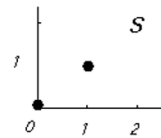
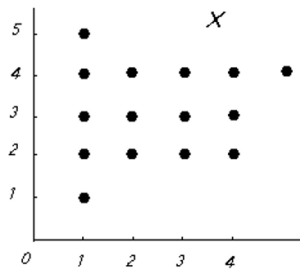


先用种子点 **1** 判断: $|I - 1| < T$ 就标成 1; 没被标到的再用种子点 **5** 判断: $|I - 5| < T$ 就标成 5; 都不满足则保留原值, 当 $T=5$ 时结果为:

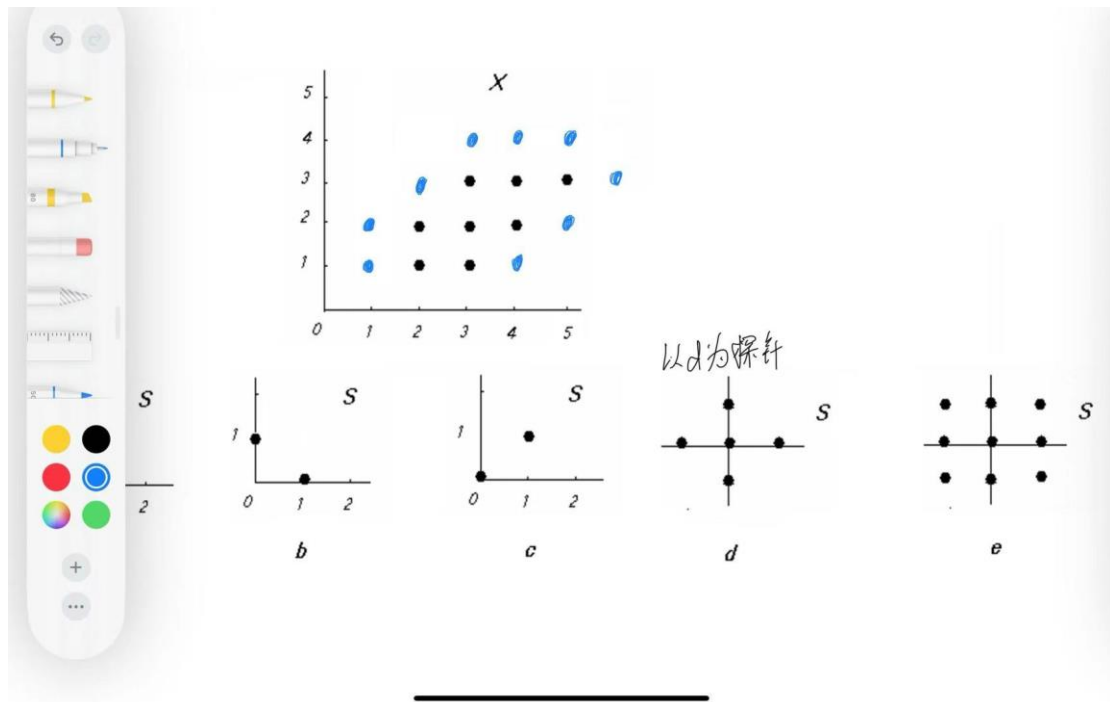
1	1	1	5	5
1	1	1	5	5
1	1	5	5	5
1	1	5	5	5
1	1	5	5	5

作业十四、腐蚀膨胀

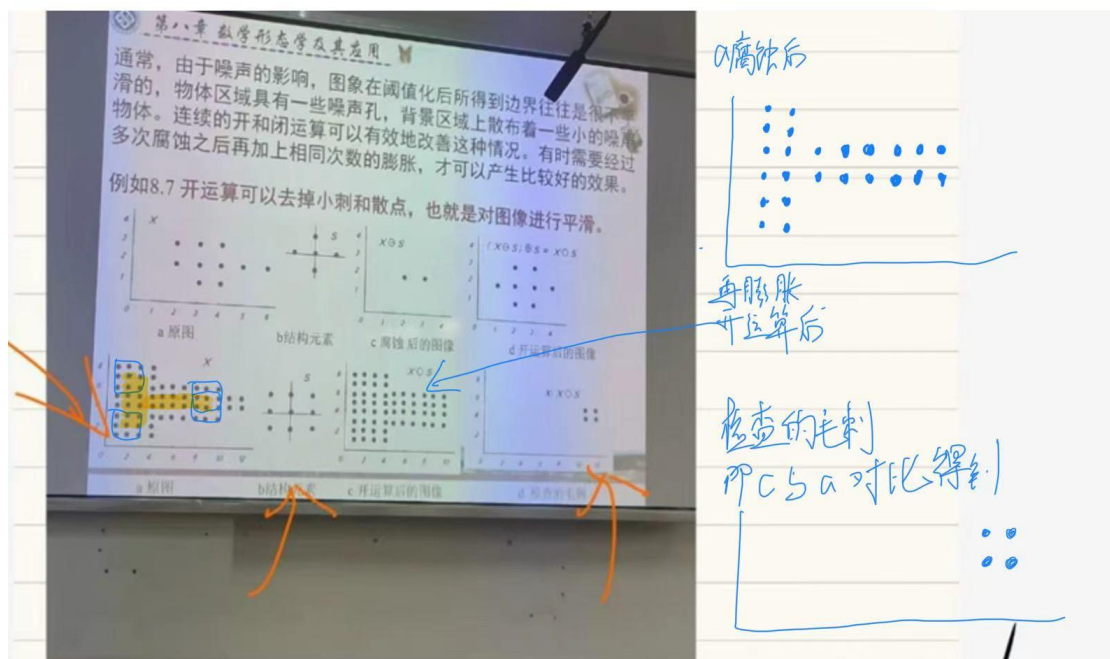
1



2



3



二值图像处理 - 基本概念

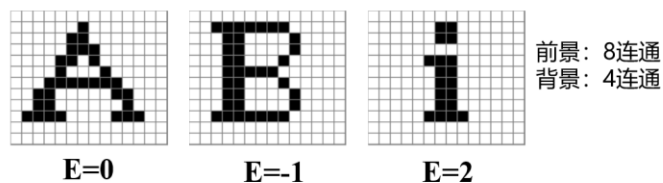


⑧ **欧拉数**：可作为识别物体的特征

定义：连通成份数减去空洞数

$$E = C - H$$

其中：C、H 为连通成份数、空洞数



特性：具有平移、旋转和比例不变特性的拓扑特征

欧拉数公式： $E = C - H$ （C：连通域个数；H：孔洞个数）

字母 C：一般是 1 个连通域、0 个孔洞

$$E = 1 - 0 = 1$$

字母 D：一般是 1 个连通域、1 个孔洞（中间那一圈是孔洞）

$$E = 1 - 1 = 0$$

所以：C 的欧拉数 = 1，D 的欧拉数 = 0。